

SC2001 Lab 2

Dijkstra's Algorithm

Graph representation • Priority queues • Empirical complexity

JAY LI YIJIE (U2540332B)

JOEL LOH QUAN YU (U2421696K)

JUSTIN HOTASI (U2310560D)

Same shortest paths. Different amounts of work.

Two implementation choices affect the work

PART A

Matrix + array

Scan possible neighbors.

Find the minimum by a linear queue scan.

$$\Theta(V^2)$$

PART B

List + min-heap

Visit existing neighbors.

Extract minima and update priorities with a heap.

$$O((V + E) \log V)$$

RESEARCH QUESTION

When is each better?

V = number of vertices

E = directed edges

R = successful updates

Compare counted work

Compare the same graph and source; change only the implementation.

Build one graph; convert it two ways

Example: $0 \rightarrow 1$ (weight 4), $0 \rightarrow 2$ (weight 2), $1 \rightarrow 2$ (weight 1)

ADJACENCY MATRIX

```
[None, 4, 2 ]  
[None, None, 1 ]  
[None, None, None]
```

Scan V cells in row u .

LIST OF ADJACENCY LISTS

```
[[ (1, 4), (2, 2) ],  
 [ (2, 1) ],  
 []]
```

Visit only u 's outgoing edges.

`matrix[u]` and `adjacency[u]` are both $O(1)$ accesses.

Actual code: graph conversions

Project_2_Dijkstra.ipynb • cell 4, lines 26–30; cell 4, lines 33–37

```
26 def to_adjacency_matrix(graph):
27     matrix = [[None] * graph.n for _ in range(graph.n)]
28     for u, v, weight in graph.edges:
29         matrix[u][v] = weight
30     return matrix

33 def to_adjacency_list(graph):
34     adjacency = [[] for _ in range(graph.n)]
35     for u, v, weight in graph.edges:
36         adjacency[u].append((v, weight))
37     return adjacency
```

Matrix allocates V^2 cells; the list allocates V lists and appends E edges.

Both implementations follow the lecture pseudocode

```
Initialize(G, source)
Q ← all vertices
while Q is not empty:
    u ← ExtractMin(Q)
    for each outgoing edge (u, v):
        Relax(u, v)
```

RELAXATION

If $\text{dist}[u] + \text{weight} < \text{dist}[v]$:

Update $\text{dist}[v]$ and $\text{parent}[v]$.

In B, also call decrease-key.

Requirements: finite, nonnegative weights; zero-weight edges are allowed.

Actual code: the shared relaxation function

Project_2_Dijkstra.ipynb • cell 6, lines 29–37

```
29 def relax(u, v, weight, dist, parent, counts):
30     # Slide 9: if dist[v] > dist[u] + weight, update parent and distance.
31     counts.relax_checks += 1
32     candidate = dist[u] + weight
33     if dist[v] > candidate:
34         parent[v], dist[v] = u, candidate
35         counts.distance_updates += 1
36         return True
37     return False
```

A successful relaxation updates both the distance and its parent.

Measure the work that explains the complexity

COUNT

Neighbor discovery

Matrix-cell probes in A.
Adjacency entries in B.

V^2 versus E

COUNT

Queue + relaxation

Minimum comparisons, heap
comparisons/swaps,
relaxation checks and
updates.

Count queue work

INTERPRET

Work, not seconds

Fixed seeds give
reproducible counts.
Constants still affect actual
runtime.

No runtime claim

Counting only relaxations would hide the difference: both attempt E relaxations.

Vary vertices and edges separately

01 • V SWEEP

Grow the graph

V: 24, 48, 96, 192, 384

Sparse: $E = 4V$

Dense: $E = \lfloor V(V - 1)/2 \rfloor$

3 seeds per setting

02 • E SWEEP

Increase density

Fix $V = 192$.

Increase E from 191 to 36,672.

Keep source = 0.

Identical inputs

03 • VALIDATE

Independent oracle

Compare against Bellman–Ford. Check reconstructed paths and invalid inputs.

57 experiment graphs

All presentation plots use linear axes and one graph per slide.

Part A: scan the queue, then scan a matrix row

STEP 01

Extract minimum

Scan the remaining array entries; choose the smallest dist.

$O(|Q|)$ per extraction

STEP 02

Discover neighbors

Read $\text{matrix}[u][v]$ for every v .
Relax only present edges.

V probes per vertex

STEP 03

Update distance

Write $\text{dist}[v]$ and $\text{parent}[v]$.
Queue reads the updated distance directly.

$O(1)$ per update

Matrix = edge discovery. Array queue = choice of the next vertex.

Actual code: Part A selects the next vertex

Project_2_Dijkstra.ipynb • cell 10, lines 1–13

```
1  def dijkstra_matrix(matrix, source):
2      # Input is a square matrix produced by to_adjacency_matrix(Graph(...)).
3      n = len(matrix)
4      counts = Counts()
5      dist, parent = initialize(n, source, counts) # Slide 16, line 1
6      Q = list(range(n)) # Line 2: all vertices
7      counts.queue_items += n
8      while Q: # Line 3
9          best = 0
10         for i in range(1, len(Q)):
11             counts.array_comparisons += 1
12             if (dist[Q[i]], Q[i]) < (dist[Q[best]], Q[best]):
13                 best = i
```

The unsorted queue scans its remaining vertices to find the minimum.

Actual code: Part A scans the matrix row

Project_2_Dijkstra.ipynb • cell 10, lines 14–23

```
14         u = Q[best]                                # Line 4: extract minimum
15         Q[best] = Q[-1]
16         Q.pop()
17         counts.extractions += 1
18         for v in range(n):                            # Line 5: discover outgoing edges
19             counts.matrix_probes += 1
20             weight = matrix[u][v]
21             if weight is not None:
22                 relax(u, v, weight, dist, parent, counts) # Line 6
23     return dist, parent, counts
```

Every extracted vertex scans all n columns, including absent edges.

Part A: two sources of quadratic work

STEP 01

Queue selection

$(V - 1) + (V - 2) + \dots + 0$
comparisons.

$$V(V - 1)/2$$

STEP 02

Matrix traversal

V rows, each with V cells.
Plus E relaxation checks and
 R successful updates.

$$V^2 + E + R$$

CONCLUSION

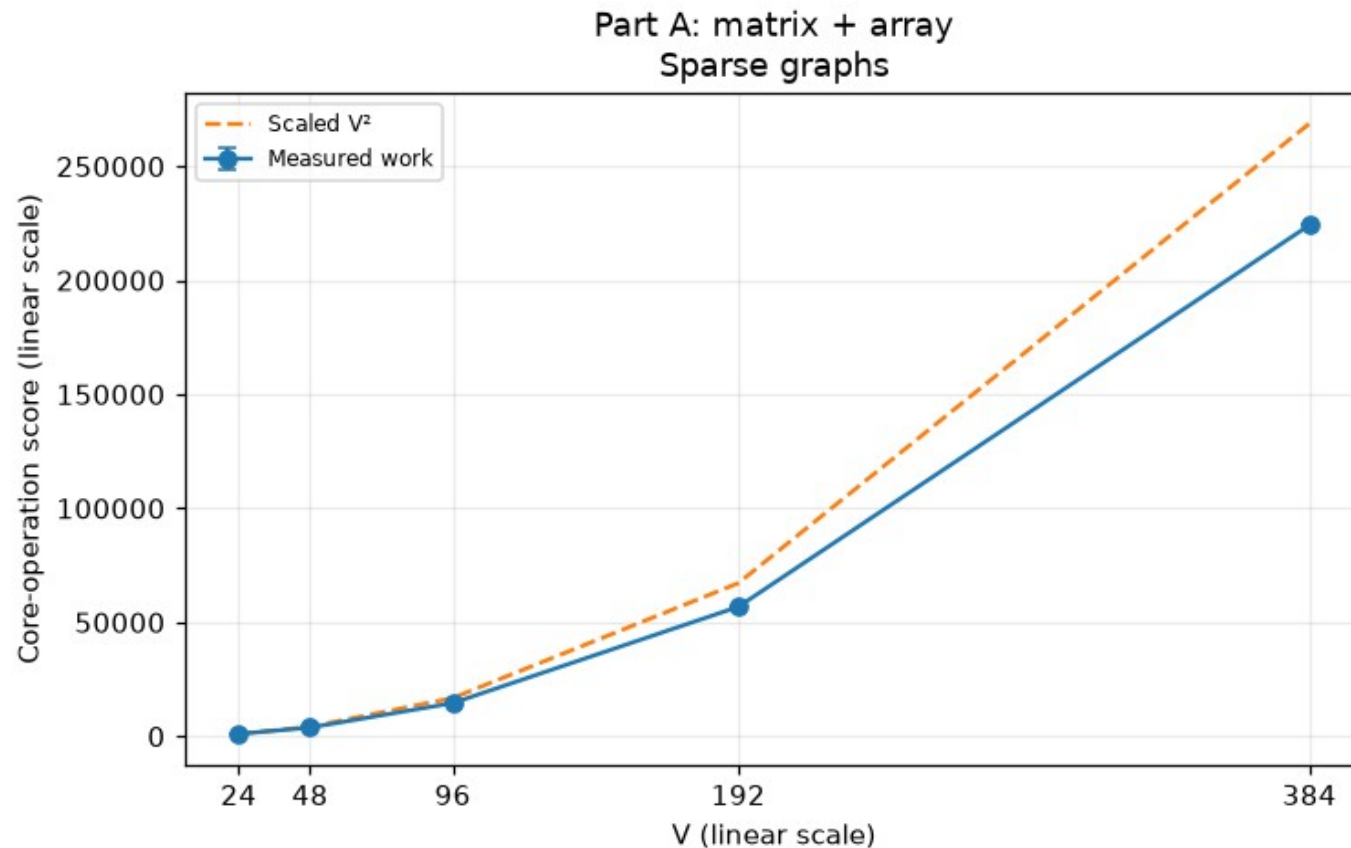
Total complexity

$E \leq V(V - 1)$, and $R \leq E$.
Matrix storage: $\Theta(V^2)$.
Auxiliary storage: $\Theta(V)$.

$$\Theta(V^2 + E) = \Theta(V^2)$$

Even a sparse graph pays for all V^2 possible-neighbor checks.

Part A: sparse graphs still incur quadratic work



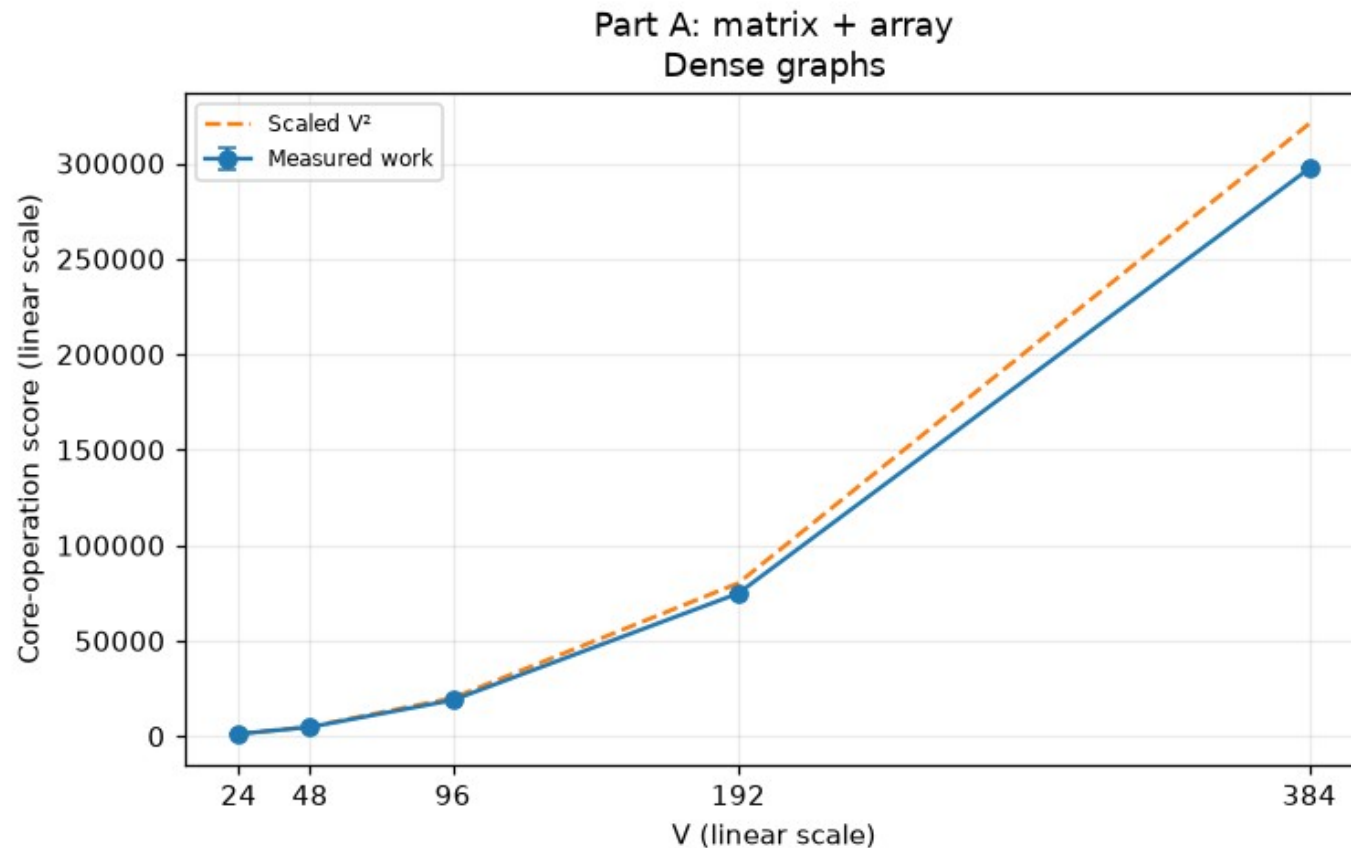
$$E = 4V$$

Most edges are absent.

The matrix and queue scans still grow quadratically.

Measured growth follows the V^2 reference.

Part A: dense graphs show the same growth order



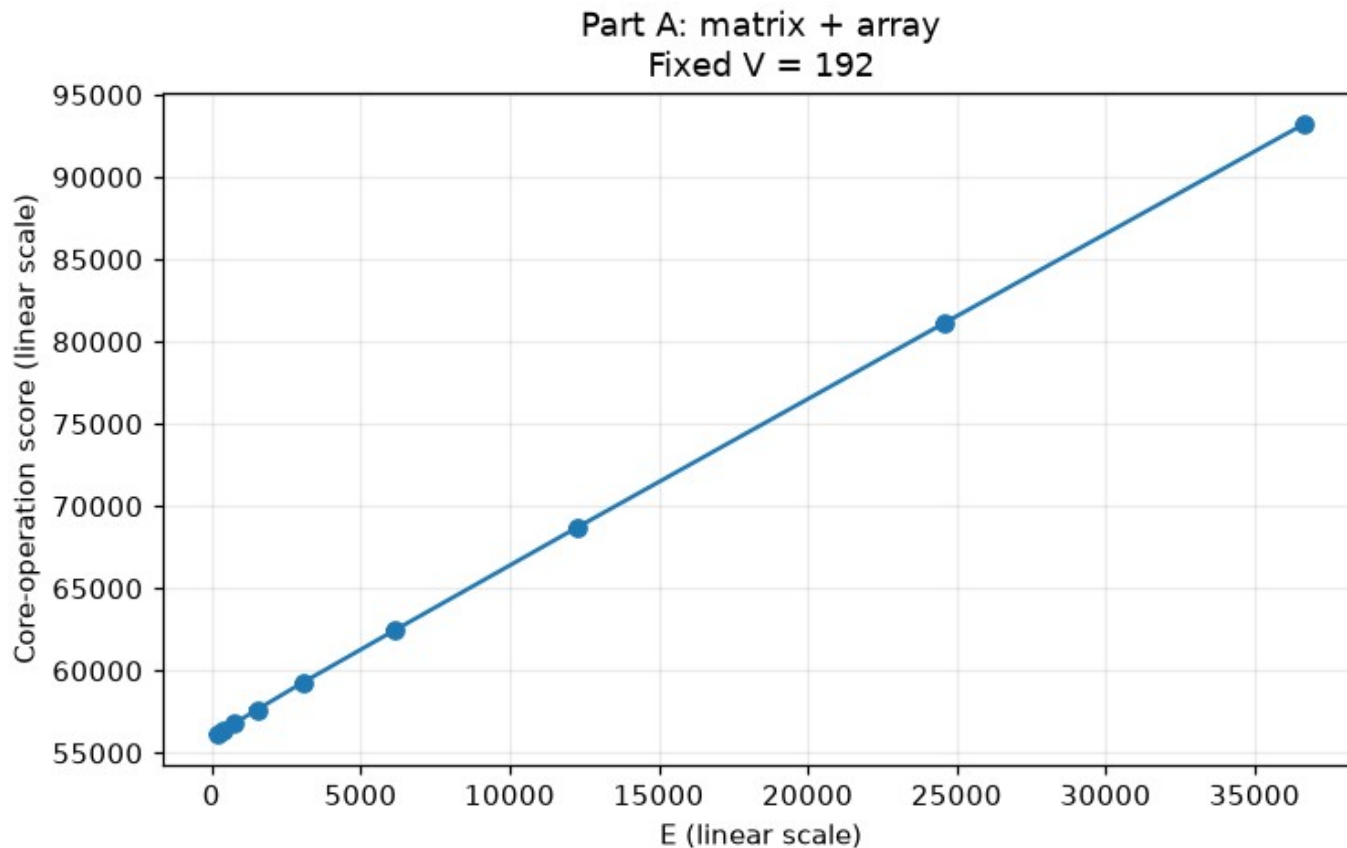
$$E \approx V^2 / 2$$

More edges add relaxation work.

The dominant growth remains quadratic.

Dense inputs add work without changing A's $\Theta(V^2)$ bound.

Part A: at fixed V, scan costs remain constant



V = 192

36,864 matrix probes.

18,336 queue
comparisons.

More edges add
relaxations.

Increasing E adds work above an already large fixed baseline.

Part B: visit existing edges and repair the heap

STEP 01

Extract minimum

Remove the root of an indexed binary min-heap.

$O(\log V)$

STEP 02

Visit neighbors

Access $\text{adjacency}[u]$ in $O(1)$. Iterate its existing edges only.

$\Theta(\text{outdegree}(u))$

STEP 03

Decrease key

After a successful relaxation, find v with $\text{position}[v]$ and sift up.

$O(\log V)$ per update

The list avoids absent edges; the heap avoids linear minimum searches.

Actual code: Part B uses adjacency lists

Project_2_Dijkstra.ipynb • cell 15, lines 60–71

```
60 def dijkstra_list(adjacency, source):
61     # Input is produced by to_adjacency_list(Graph(...)).
62     counts = Counts()
63     dist, parent = initialize(len(adjacency), source, counts) # Line 1
64     Q = IndexedMinHeap(dist, counts) # Line 2
65     while Q: # Line 3
66         u = Q.extract_min() # Line 4
67         for v, weight in adjacency[u]: # Line 5
68             counts.list_entries += 1
69             if relax(u, v, weight, dist, parent, counts): # Line 6
70                 Q.decrease_key(v) # Keep Q consistent with changed dist[v].
71     return dist, parent, counts
```

Only existing neighbors are visited; successful updates call decrease-key.

Actual code: extracting the heap minimum

Project_2_Dijkstra.ipynb • cell 15, lines 36–45

```
36     def extract_min(self):
37         self.counts.extractions += 1
38         result = self.heap[0]
39         last = self.heap.pop()
40         self.position[result] = -1
41         if self.heap:
42             self.heap[0] = last
43             self.position[last] = 0
44             self._sift_down(0)
45         return result
```

Replace the root with the last entry, then sift down to restore heap order.

Actual code: decreasing a heap priority

Project_2_Dijkstra.ipynb • cell 15, lines 47–57

```
47     def decrease_key(self, vertex):
48         # The caller has just decreased dist[vertex]. Restore heap order.
49         self.counts.decrease_keys += 1
50         i = self.position[vertex]
51         assert i >= 0, 'An extracted vertex cannot improve with nonnegative weights.'
52         while i > 0:
53             parent = (i - 1) // 2
54             if not self._less(i, parent):
55                 break
56             self._swap(i, parent)
57             i = parent
```

position[vertex] finds the heap entry in $O(1)$; sift-up takes $O(\log V)$.

Part B: edge visits plus heap maintenance

STEP 01

Visit existing edges

Each adjacency entry is read once.

Each edge is relaxed once.

$$\Theta(E)$$

STEP 02

Maintain the heap

V extract-min calls.

R decrease-key calls.

Heap construction is $O(V)$.

$$O((V + R) \log V)$$

CONCLUSION

Worst-case bound

Use $R \leq E$.

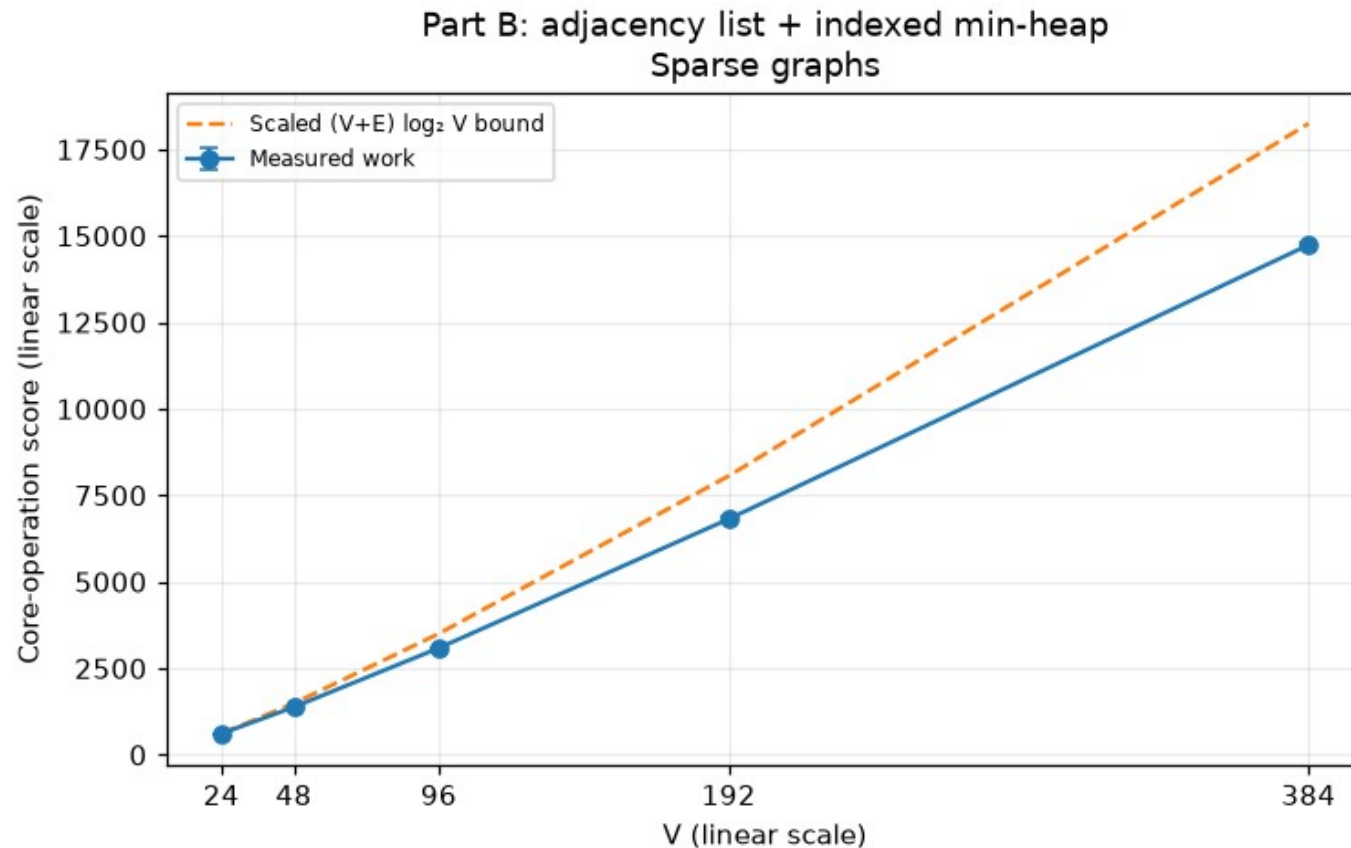
Graph space: $\Theta(V + E)$.

Auxiliary space: $\Theta(V)$.

$$O((V + E) \log V)$$

An upper bound does not say every edge causes a logarithmic-cost update.

Part B: sparse graphs benefit from both choices



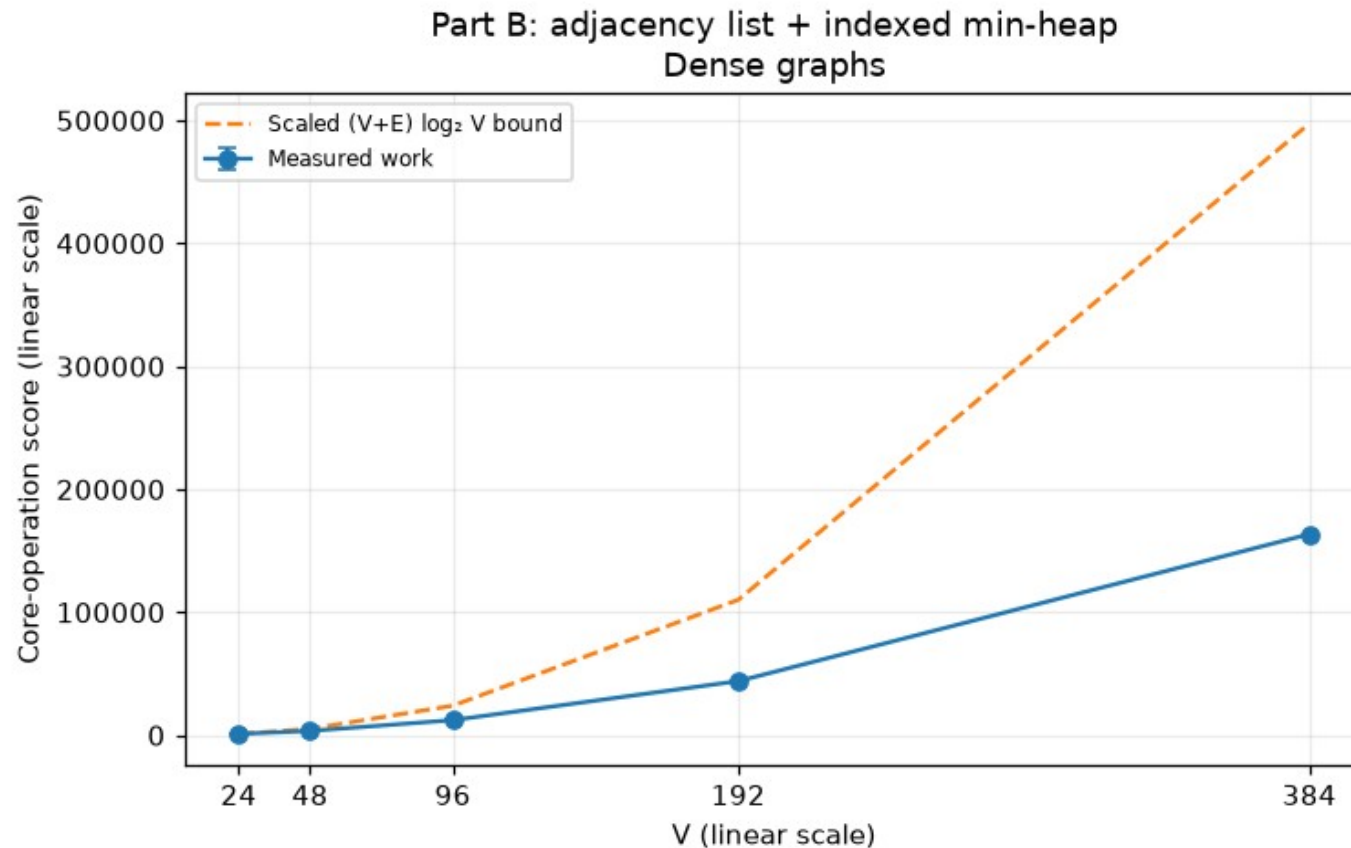
$O(V \log V)$

Only $4V$ edges are visited.

The heap replaces repeated linear minimum searches.

Sparse inputs avoid both quadratic costs found in Part A.

Part B: a dense upper bound can be pessimistic



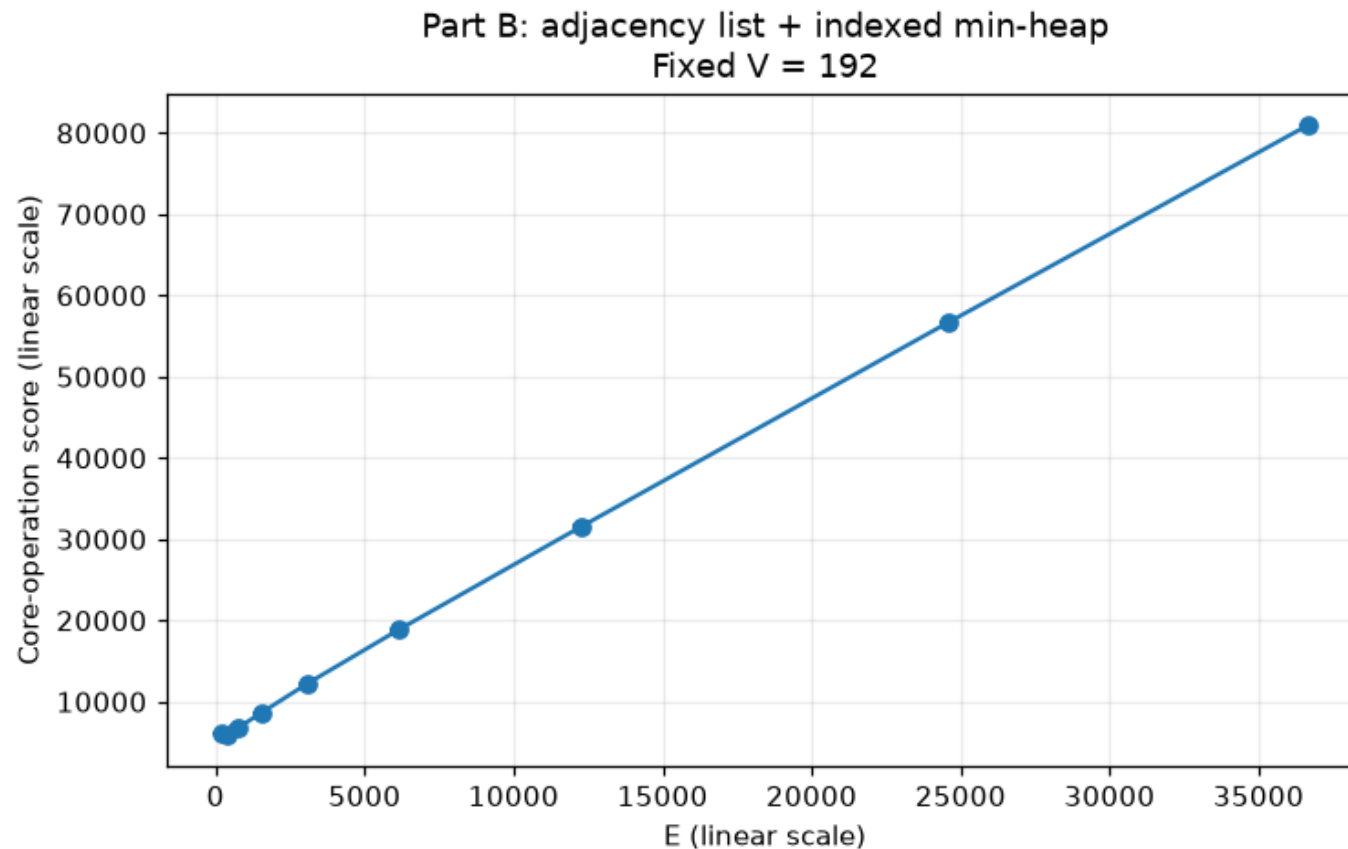
R can be $\ll$ E

Every edge is checked.

Only successful improvements trigger decrease-key.

Observed work can grow closer to V^2 than $V^2 \log V$.

Part B: adding edges directly adds list visits



$V = 192$

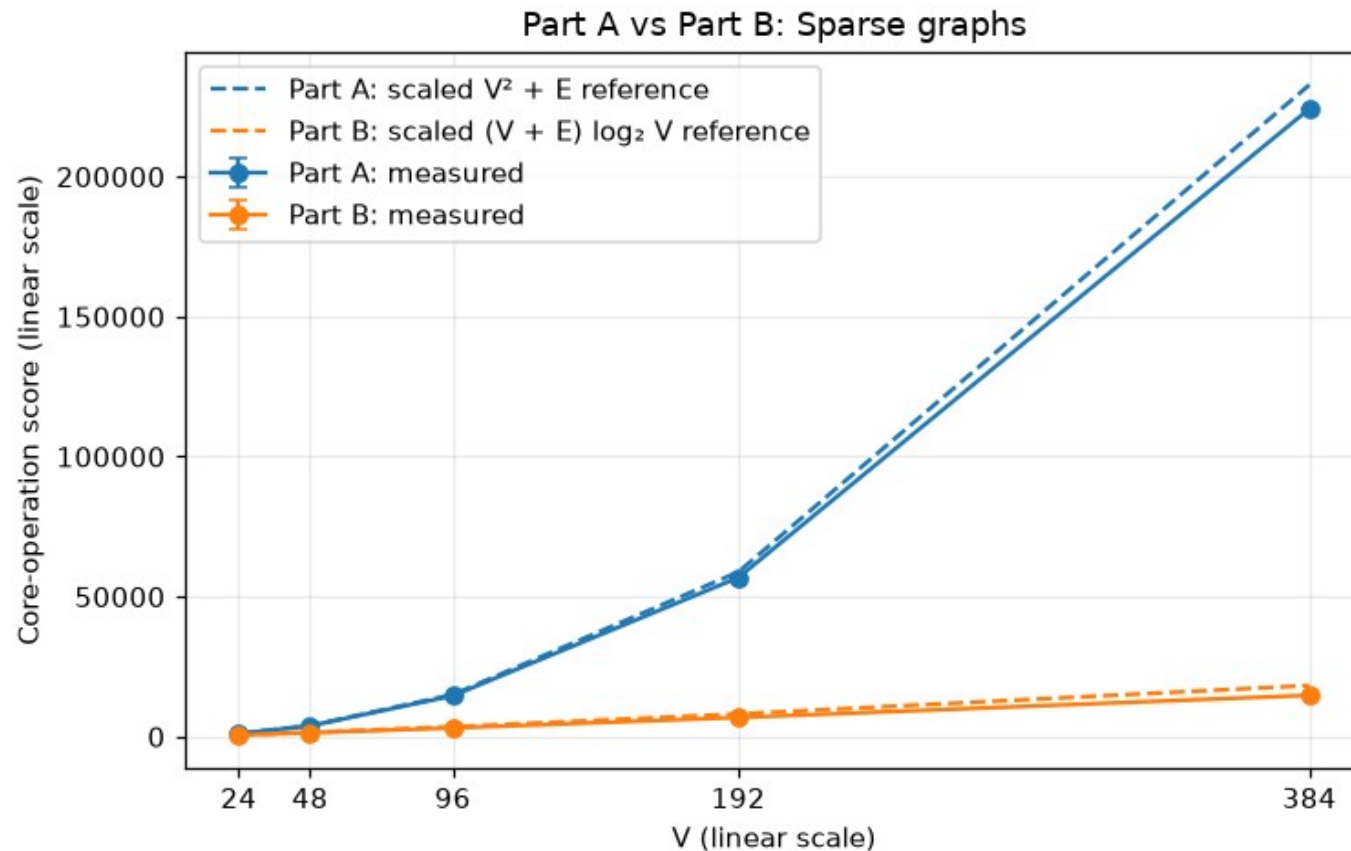
E list-entry visits.

E relaxation checks.

Heap work depends on
successful updates.

At fixed V , edge traversal grows linearly with E .

Part C: Part B uses much less work on sparse inputs



$$A / B = 15.21$$

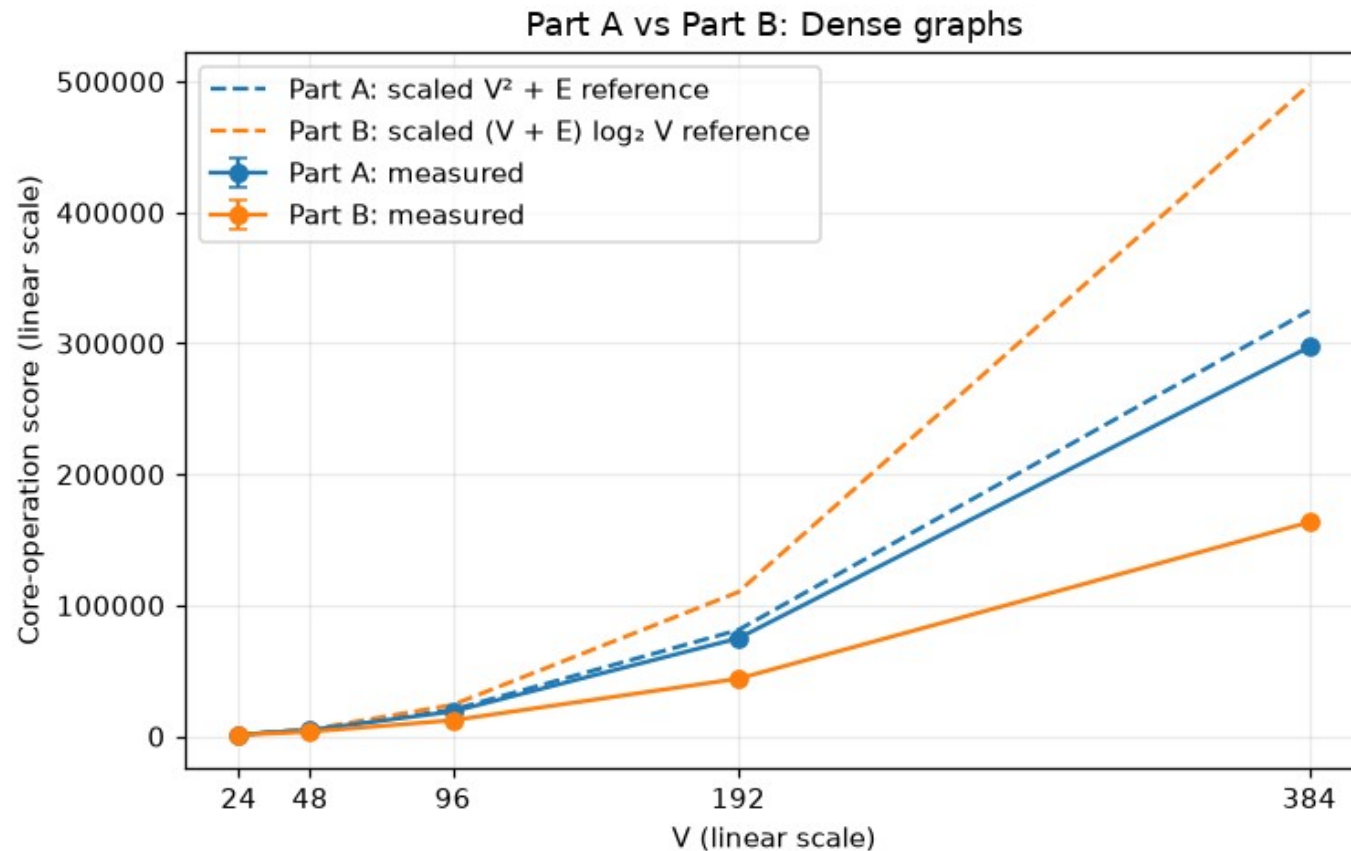
At $V = 384$, $E = 1,536$:

A: 224,192 operations

B: 14,737 operations

This is a work-score ratio, not a runtime speedup.

Part C: Part B also wins these random dense inputs



$$A / B = 1.82$$

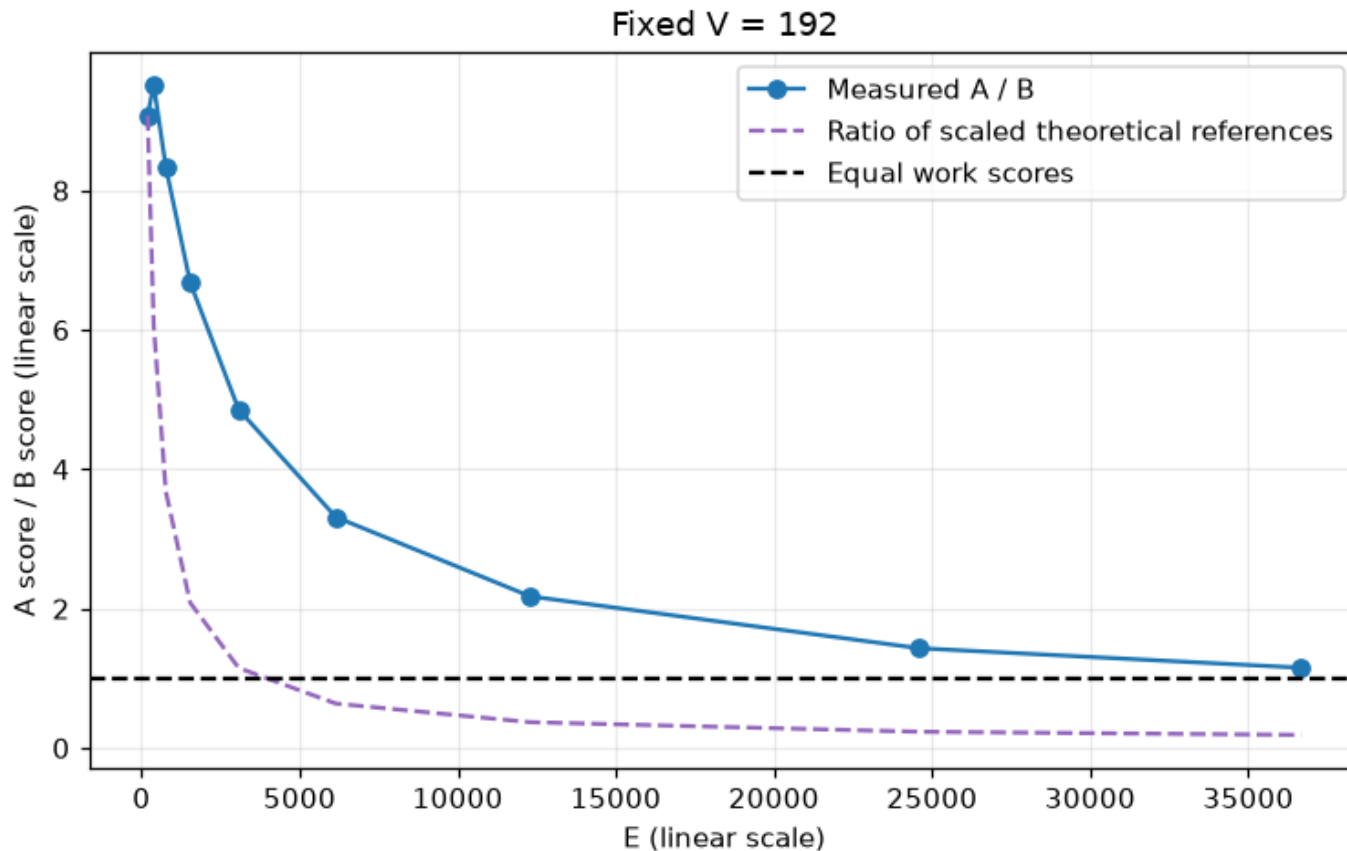
At $V = 384$, $E = 73,536$:

A: 297,284 operations

B: 163,496 operations

A's better worst-case bound does not guarantee a win on every dense graph.

Part C: why the ratio falls as E increases



Read A / B

Above 1: B uses less work.

Below 1: A uses less work.

Crossing 1 is not required.

A's fixed scan costs are large; B's edge-dependent work catches up.

Frequent updates can make Part A preferable

Complete directed graph • $V = 384$ • $E = 147,072$ • $R = E / 2$

Part A



442,752

Part B



522,916

Counted operations; bars start at zero.

Part A uses 15.3% fewer counted operations when many priorities improve.

Separate the representation from the priority queue

Choice	Matrix / array	List / heap
Neighbor discovery	Matrix: V^2 cells	List: E entries
Minimum selection	Array: $\Theta(V^2)$ total	Heap: $O(V \log V)$ total
Priority updates	Array: $O(1)$ each	Heap: $O(\log V)$ each
Specific edge lookup	Matrix: $O(1)$	List: $O(\text{outdegree}(u))$

List + array also gives $\Theta(V^2 + E)$: a matrix is not essential for that bound.

Choose based on sparsity and update behavior

- 01** Sparse graphs: list + heap avoids wasted scans and saves space.
- 02** Dense graphs: array updates give a strong worst-case bound; a win is not automatic.
- 03** Report what was measured: core-operation counts, not CPU speedups.

Thank you — questions?

Exact counters connect theory to the measurements

Part A

$$C_A = 3V + V(V - 1)/2 + V^2 + E + R$$

Part B

$$C_B = 3V + 2E + 2R + H_c + H_s$$

R: successful relaxations H_c: heap comparisons H_s: heap swaps

Generation, conversion, plotting and validation are outside these counts.

Validation, scope and sources

VALIDATION

Independent checks

Bellman–Ford distances.

Reconstructed path weights.

Zero weights, ties and disconnected graphs.

All checks passed

LIMITATIONS

Controlled experiments

Three seeds per setting.

Moderate input sizes.

Counts $\neq$ elapsed time.

References are scaled.

No universal crossover

SOURCES

Project materials

Project 2.pdf: parts a–c

06_ShortestPath.pdf: slides 9–10, 13, 15–18

Project_2_Dijkstra.ipynb

Notebook results

Speaker notes include the detailed explanations and assumptions.